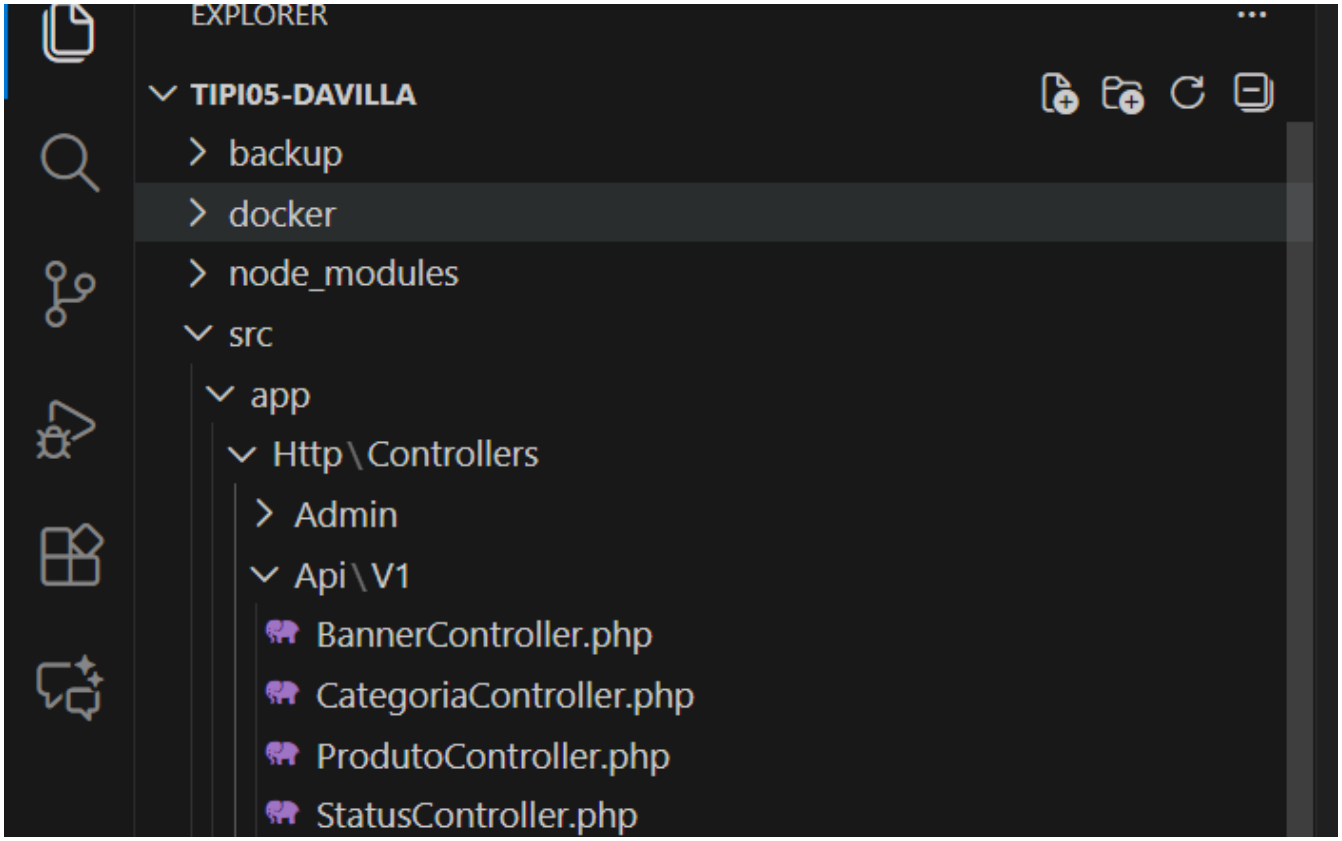
Criação da API

Código-fonte do projeto no GitHub: https://github.com/allepalmeira/tipi05-davilla

1. Controllers da API

Os Controllers concentram a lógica de entrada da API. Eles recebem a requisição, consultam os dados necessários e montam a resposta que será enviada ao cliente. Para manter a estrutura organizada e permitir evolução futura, os Controllers foram separados dentro de Api/V1. O conteúdo completo dos Controllers está versionado no GitHub do projeto, no caminho src/app/Http/Controllers/Api/V1/.

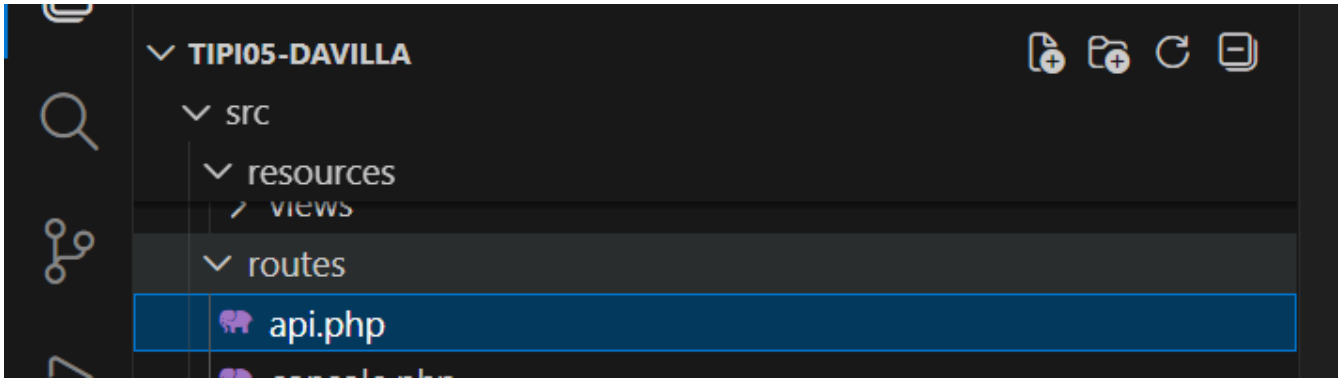


1.1 Comandos utilizados após criar as pastas:

```
docker compose exec --user "$(id -u):$(id -g)" php php artisan make:controller Api/V1/StatusController
docker compose exec --user "$(id -u):$(id -g)" php php artisan make:controller Api/V1/BannerController
docker compose exec --user "$(id -u):$(id -g)" php php artisan make:controller Api/V1/CategoriaController
docker compose exec --user "$(id -u):$(id -g)" php php artisan make:controller Api/V1/ProdutoController
```

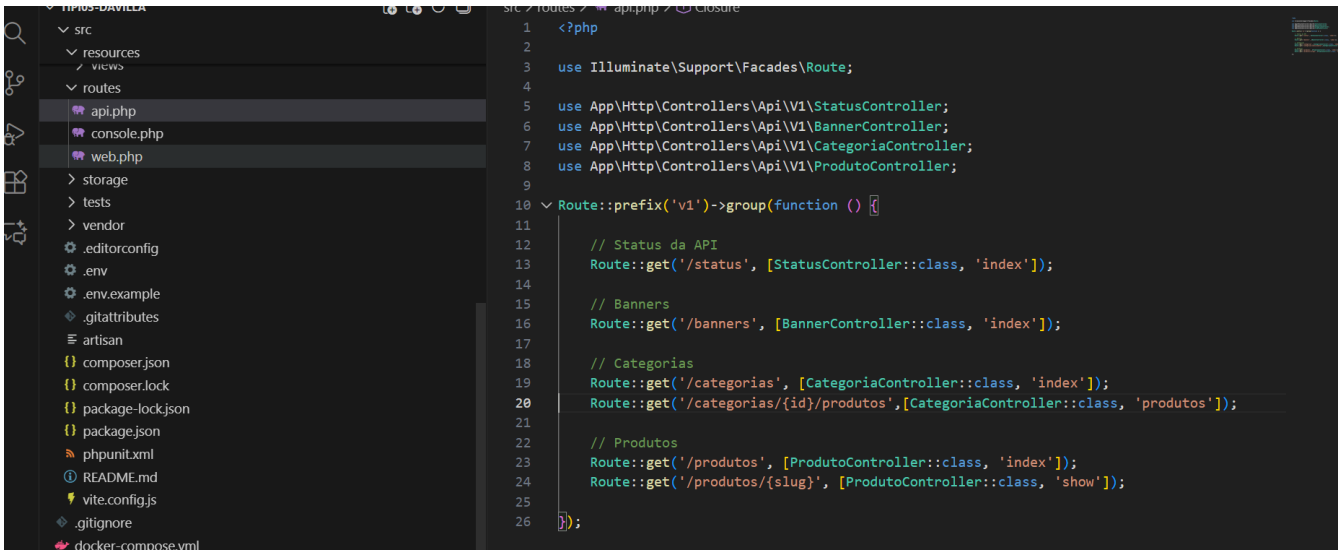
2. Rotas da API

As rotas definem qual URL será disponibilizada e qual Controller deverá atender cada requisição. Criamos um arquivo específico para a API: src/routes/api.php.



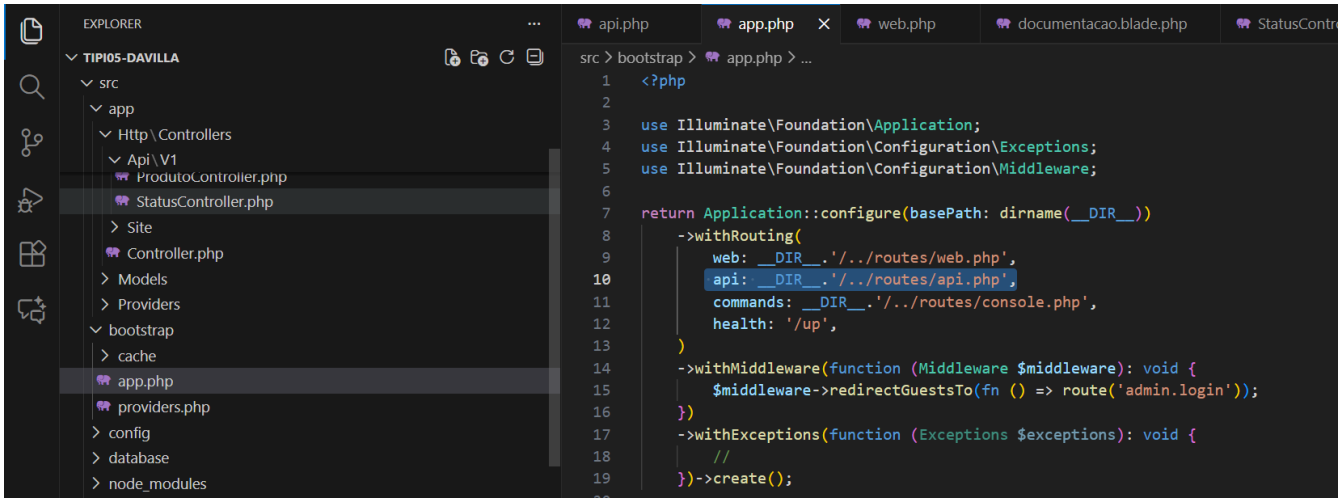
Dentro de routes/api.php, usamos o prefixo v1. O Laravel acrescenta automaticamente o prefixo /api quando esse arquivo é registrado em bootstrap/app.php. Assim, os endpoints passam a seguir o padrão /api/v1/....

2.1 Estrutura simplificada de routes/api.php



3. Registro da API em bootstrap/app.php

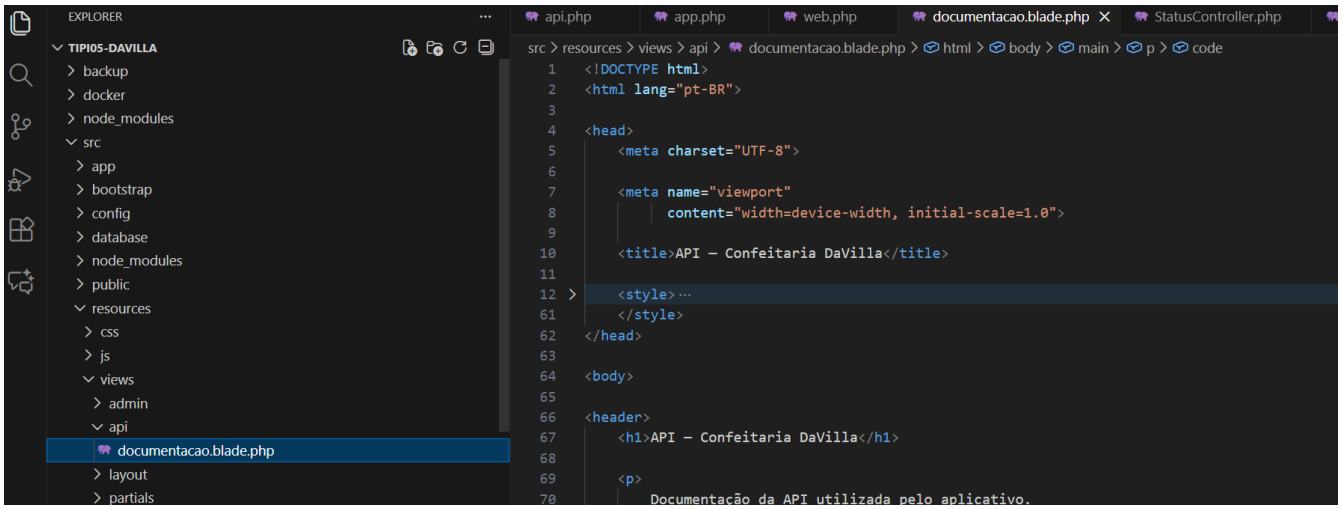
Criar routes/api.php não é suficiente. O Laravel precisa saber que esse arquivo deve ser carregado. Por isso alteramos src/bootstrap/app.php. Antes, o projeto registrava apenas as rotas web e de console; acrescentamos a propriedade api.



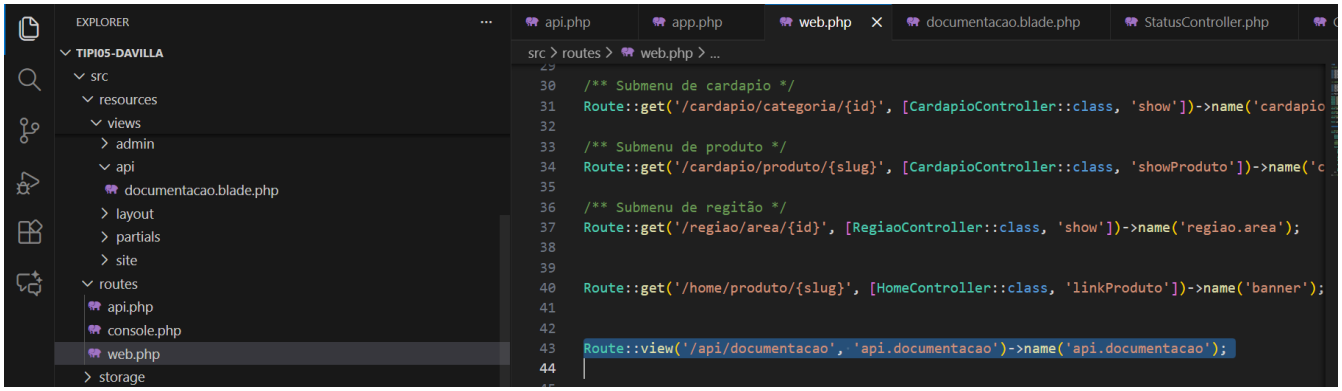
O bootstrap é parte da inicialização da aplicação. Durante o carregamento do Laravel, ele define quais arquivos de rotas farão parte da aplicação.

4. View da documentação da API

A API retorna JSON, mas também criamos uma página visual para documentar os endpoints. Essa página é uma View Blade e, por isso, pertence ao fluxo Web.



A rota da documentação fica em routes/web.php, ela conecta a URL /api/documentacao.



A documentação pode ser acessada localmente em: **http://localhost:8080/api/documentacao**

4.1 Diferença entre API e View

Fluxo	Exemplo	Resposta
API	/api/v1/produtos	JSON para aplicativo ou outro sistema
WEB	/api/documentacao	HTML gerado por Blade para leitura humana

5. Validação realizada

Depois de criar os Controllers, as rotas e registrar a API no bootstrap, validamos a lista de rotas e testamos o primeiro endpoint de status.

```
docker compose exec php php artisan route:clear

docker compose exec php php artisan route:list --path=api/v1

curl http://localhost:8080/api/v1/status
```

5.1 Validação no navegador com JSON Viewer Pro

Também realizamos a validação diretamente no navegador em http://localhost:8080/api/v1/status utilizando a extensão JSON Viewer Pro. A extensão organiza e formata a resposta JSON, facilitando a leitura da estrutura retornada pela API e a conferência dos campos success, data, aplicacao, api e status.



6. Próximo passo: API com credenciais

Com a base pública da API funcionando, o próximo passo será implementar a API com credenciais para identificar e autenticar o cliente. A partir da autenticação, poderemos proteger os endpoints que dependem de um usuário identificado e avançar para recursos como favoritos, endereços, depoimentos, cupons e pedidos.